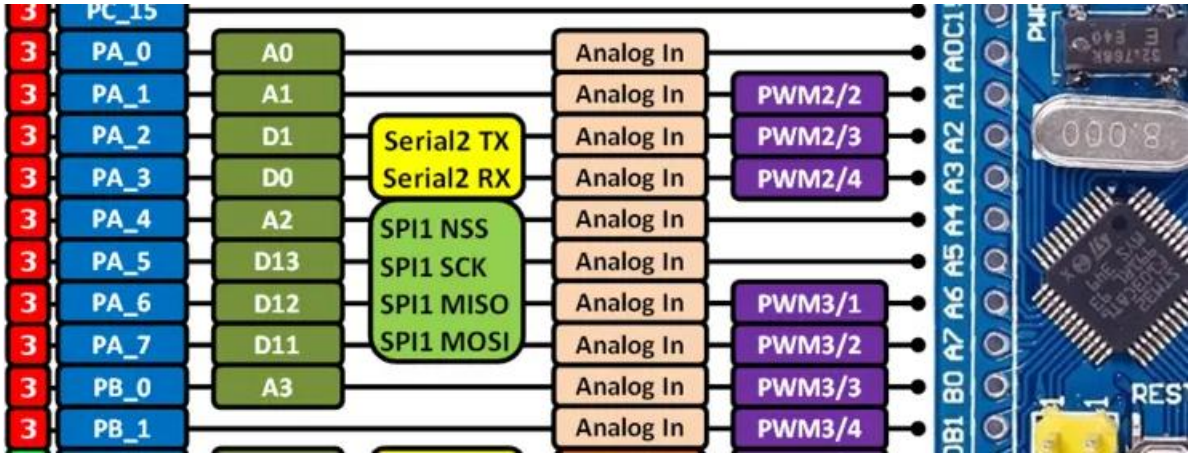
Conversão Analógico-Digital no STM32F103C8T6

Características do ADC

- 2 conversores ADC de 12 bits (ADC1 e ADC2)
- Até 16 canais externos
- Taxa de amostragem de até 1 MHz
- Tensão de referência: 0 a 3,3V (padrão)
- Modos de conversão: contínuo, único ou por injeção



Criando o Projeto

1. Abra STM32CubeIDE
2. Crie novo projeto com STM32F103C8T6
3. Configure clock e periféricos
4. Inicialize todos os periféricos no modo padrão

Configuração do ADC

- Ative ADC1 na aba "Analog"

- Parâmetros:

- Modo: Single/Continuous

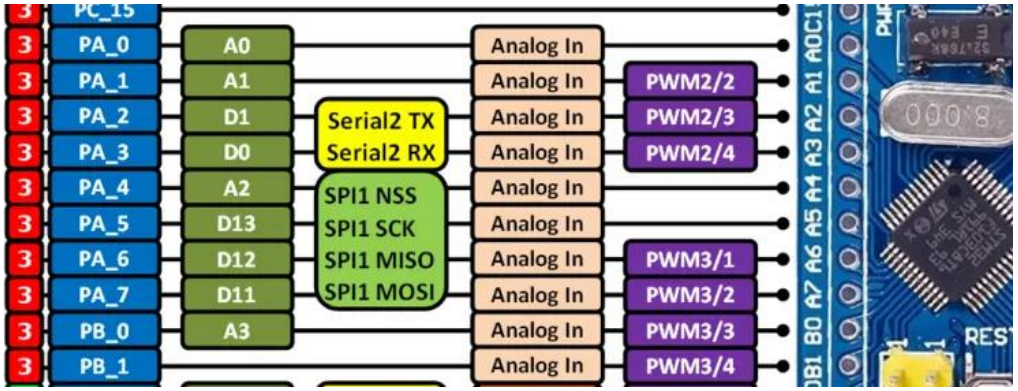
- Resolução: 12 bits

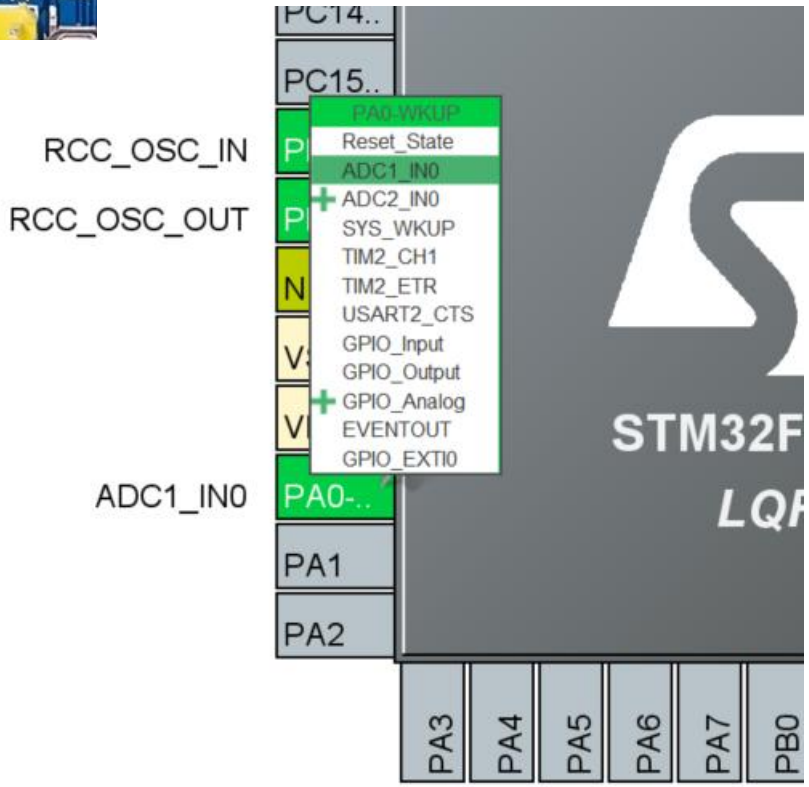
- Alinhamento: à direita

- N° de conversões: 1

(caso aquisite apenas 1 ADC)

Seleção de Pinos

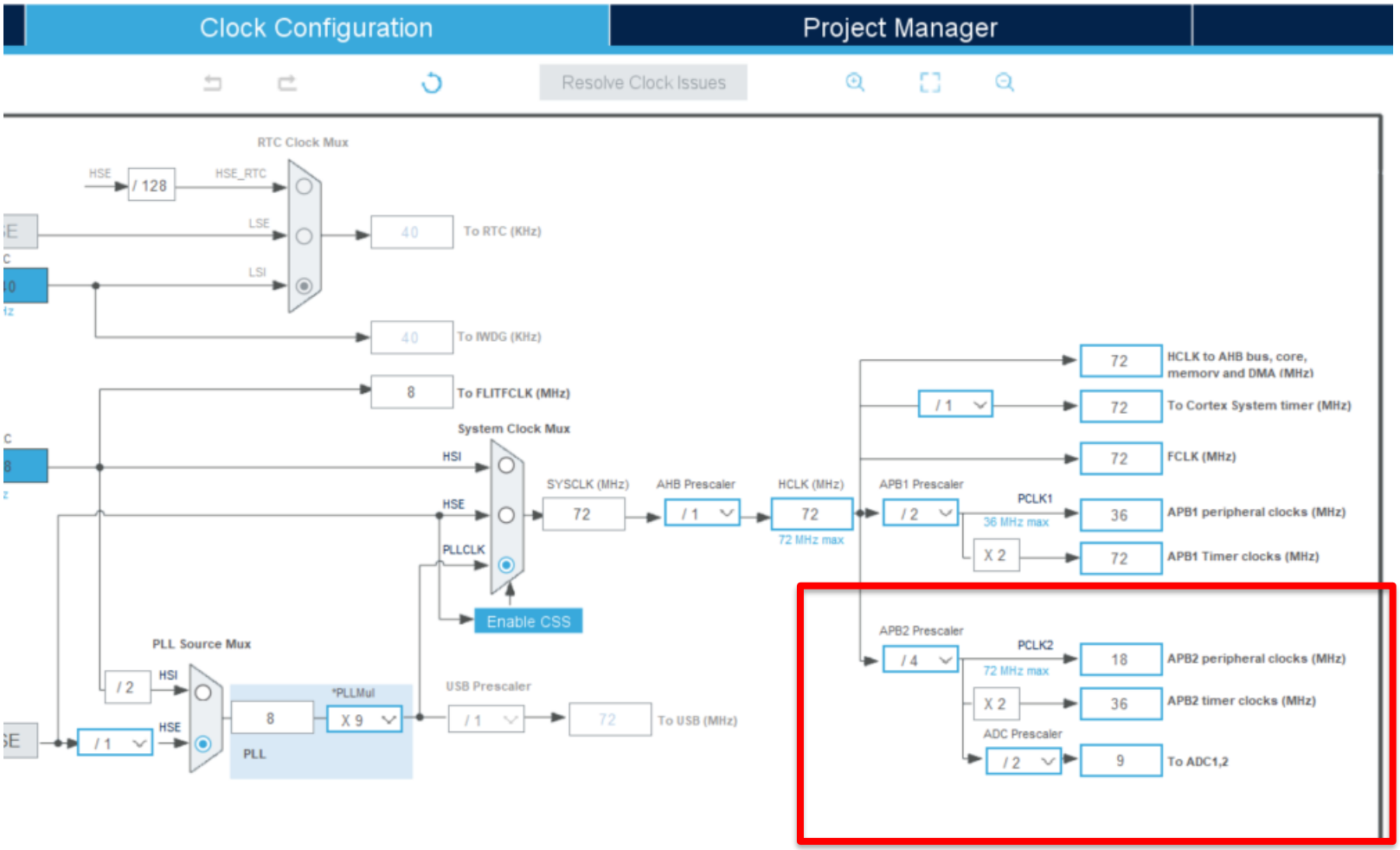




Escolha o pino desejado entre as opções disponíveis

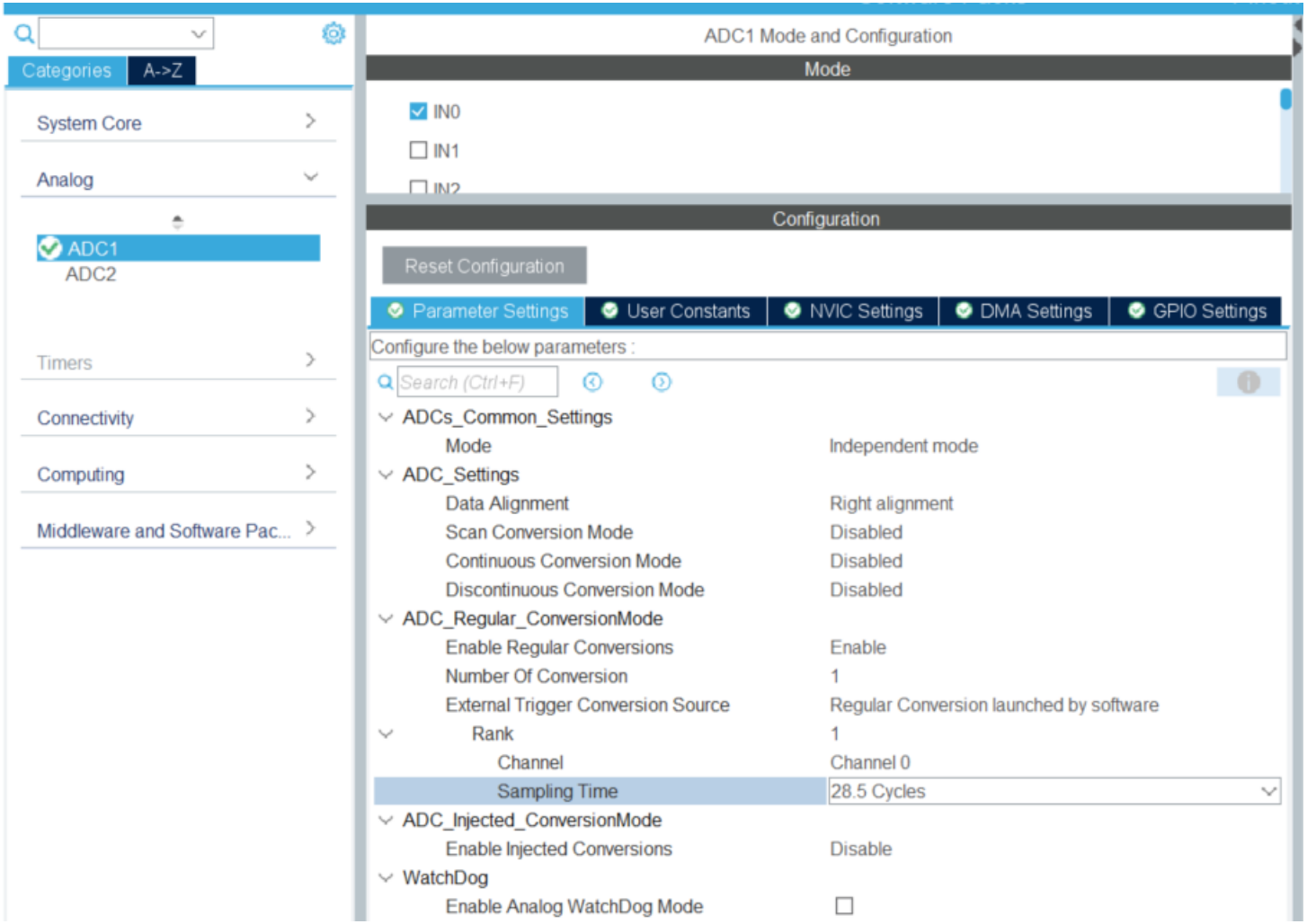
Escolha entre configurar como: ADC1 ou ADC2

Configuração do Clock



Recomendação: ADC clock máximo de 14MHz

Leitura no Modo Polling



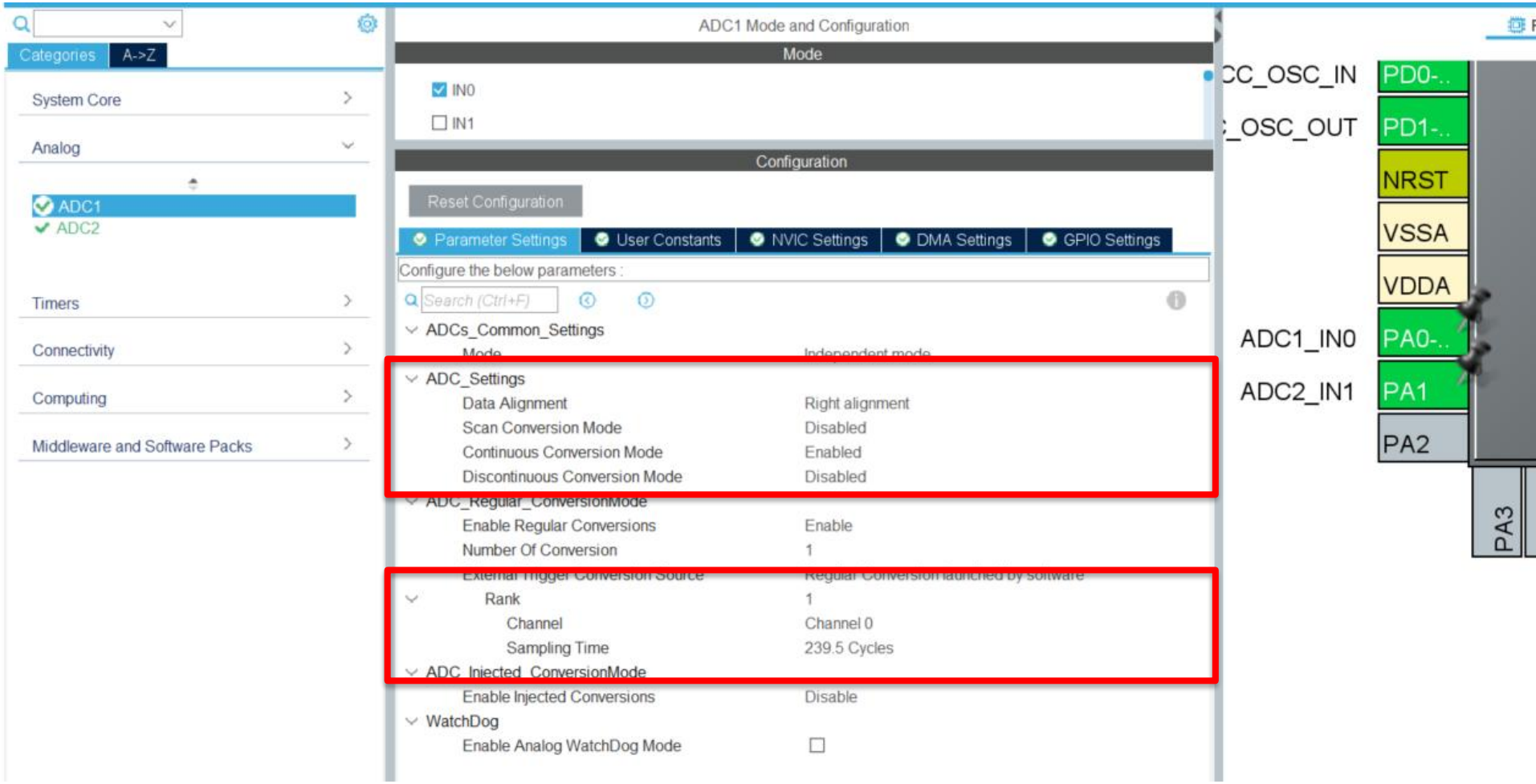
Configure o tempo de amostragem (geralmente 28.5 ciclos)

Leitura no Modo Polling

```
HAL_ADC_Start( &hadc1); // Inicia a conversão ADC
HAL_ADC_PollForConversion(&hadc1, 10);
// Espera a conversão ser concluída
valorADC = HAL_ADC_GetValue(&hadc1);
// Lê o valor convertido - Devolve um numero entre 0 e 4096
HAL_ADC_Stop(&hadc1); //Desabilita o periférico
```

```c
// Variável para armazenar o valor convertido
 uint16_t valorADC;

 HAL_ADC_Stop(&hadc1);
 HAL_ADCEx_Calibration_Start(&hadc1);


/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  HAL_ADC_Start( &hadc1); // Inicia a conversão ADC
  HAL_ADC_PollForConversion(&hadc1, 10);// Espera a conversão ser concluída
  valorADC = HAL_ADC_GetValue(&hadc1);// Lê o valor convertido - Devolve um nume
  HAL_ADC_Stop(&hadc1);
```

Leitura no Modo Continuous



Leitura no Modo Continuous

HAL_ADC_Start(&hadc1);

- Inicia o modulo ADC na “void main”.

adcValue = HAL_ADC_GetValue(&hadc1);

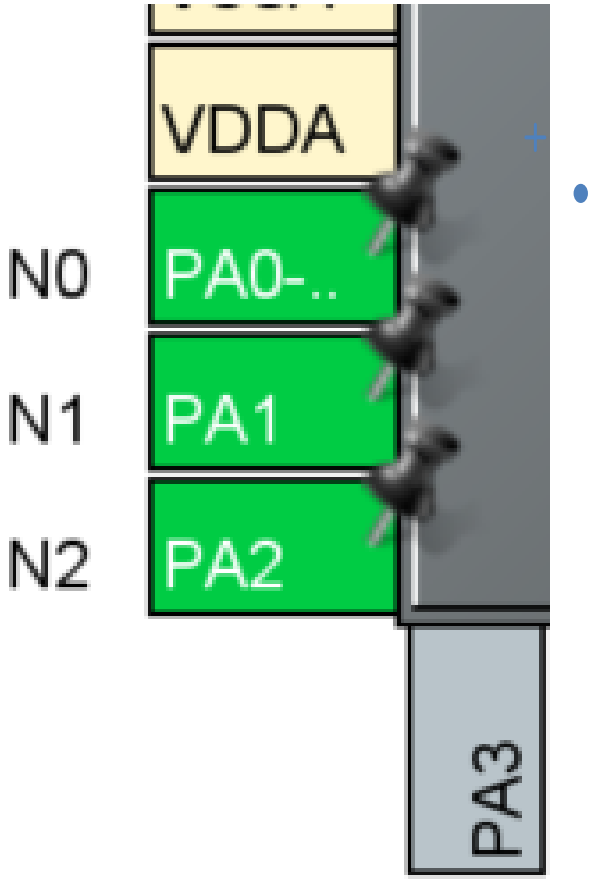
- No loop While, ou chamada de função desejada retorna o valor de conversão armazenado.

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_ADC1_Init();
HAL_ADC_Start(&hadc1);

LCD16X2_Init(MyLCD);
LCD16X2_Clear(MyLCD);
LCD16X2_Set_Cursor(MyLCD, 1, 1);
LCD16X2_Write_String(MyLCD, "PROF Mozart");
HAL_Delay(2000);

while (1)
{
  adcValue = HAL_ADC_GetValue(&hadc1);
```

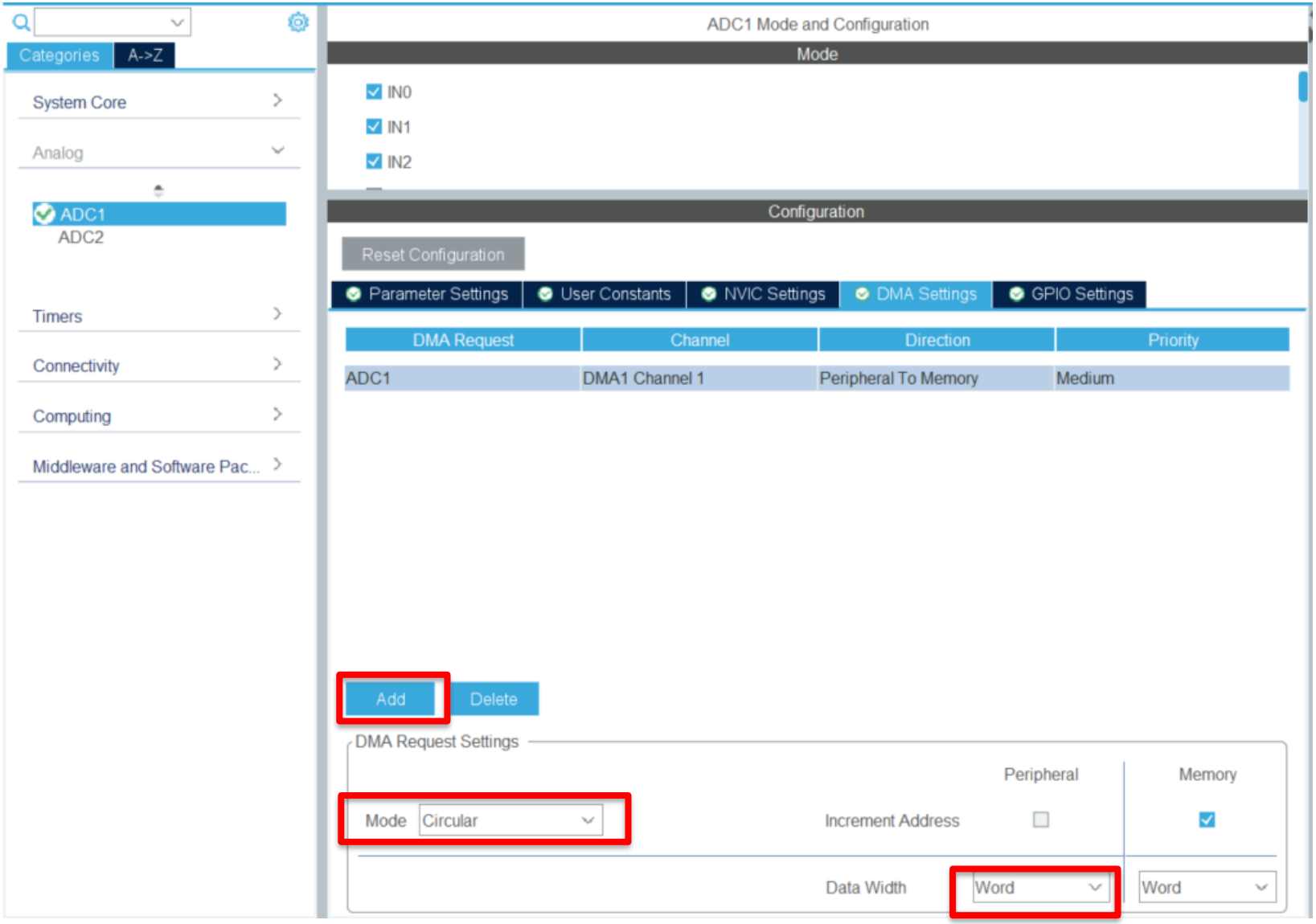
Leitura com DMA

Indicada para a leitura rápida de varias pinos, atribuindo uma taxa de amostragem padão.

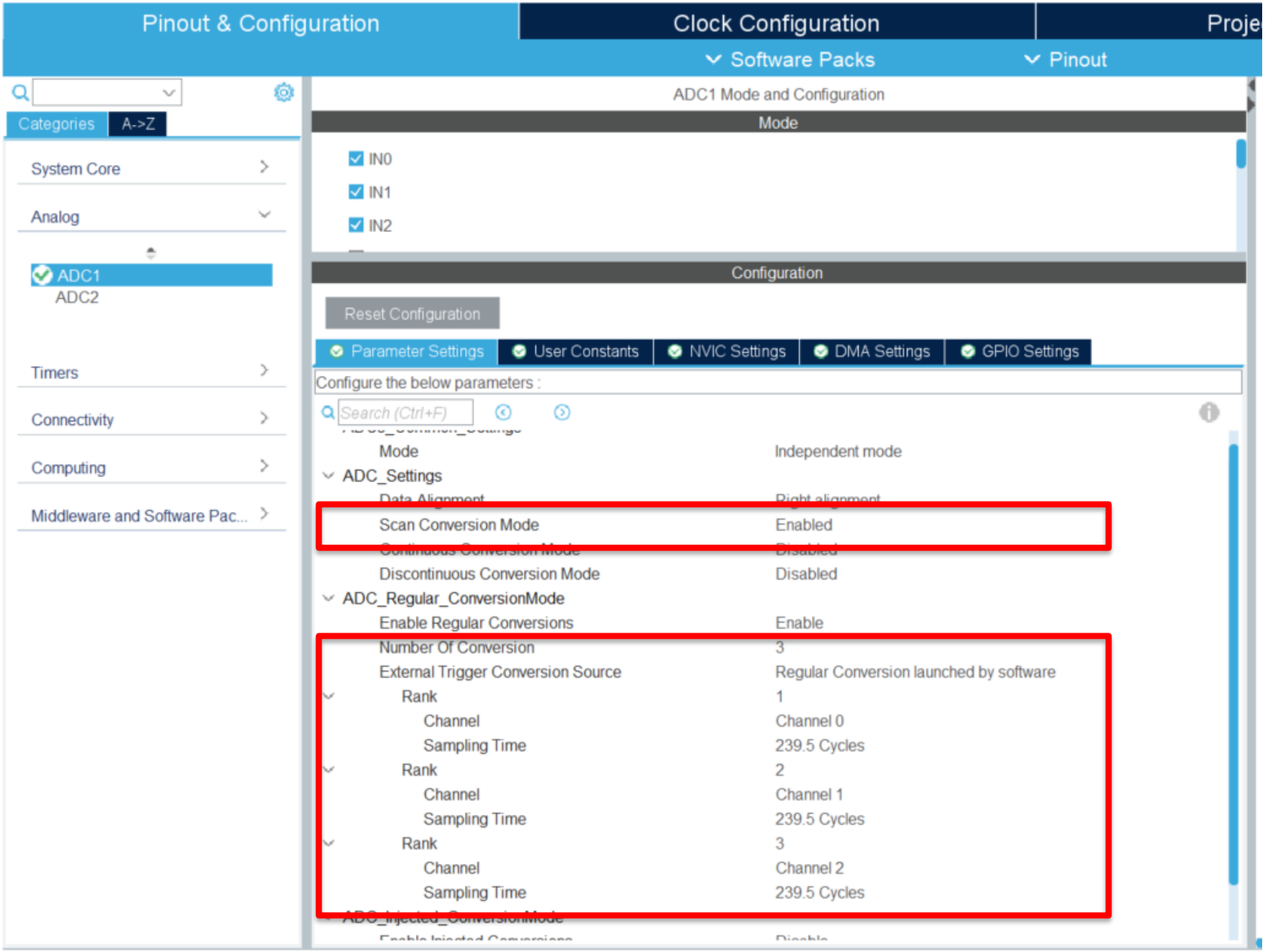
Ideal para a leitura de múltiplos sensores.



Leitura com DMA



Leitura com DMA



Leitura com DMA

```
uint32_t adc_buffer[2];
```

Crie um vetor para armazenar as conversões.

```
HAL_ADC_Start_DMA(&hadc1, adc_buffer, 2);
```

Chame a função para inicializar a conversão

Conversão para Tensão

```
uint16_t adcValue; // Variável para armazenar o resultado da conversão analógica para digital
// Varia de 0 a 4095 (12bits)
float Volt; //Variável para armazenar o valor em Volt
//Varia de 0 a 3.3 Volts
Volt = adcValue *( 3.3 / 4095);
```

Boas práticas

- Calibre o ADC antes do uso: HAL_ADCEx_Calibration_Start(&hadc1);
- Use capacitores de desacoplamento próximos aos pinos analógicos
- Mantenha sinais digitais longe dos analógicos
- Para sinais ruidosos, considere:
  - Média de múltiplas leituras
  - Filtros digitais simples

Referências

- STM32F103xx Reference Manual (RM0008)
- STM32F103 Datasheet
- STM32CubeF1 HAL Documentation
- STMicroelectronics Application Notes (AN3116, AN2834)